

Programmation Fonctionnelle

TD n° 5 : Calcul propositionnel en OCaml

```
type formula =
  | Prop of string
  | Neg of formula
  | And of formula * formula
  | Or of formula * formula
```

Exercice 1 : Écrire une fonction `string_of_formula : formula -> string` qui, étant donnée une formule, renvoie une chaîne de caractères représentant cette formule écrite avec des parenthèses encadrant toutes les sous-formules de connecteur binaire.

Par exemple, pour `Or (Prop "a", Prop "b")`, on obtient `"(a Or b)"`, et pour `And (Prop "p", Neg (Or (Prop "q", Prop "r")))`, on obtient `"(p And Neg (q Or r))"`.

Exercice 2 : Écrire une fonction `list_of_props : formula -> string list` qui, étant donnée une formule, renvoie une liste sans répétitions de tous ses noms de propositions. Par exemple, `list_of_props (And (Or (Prop "p", Prop "q"), Prop "q"))` donnera `["p"; "q"]`, à l'ordre des éléments près.

Pour construire l'union sans répétitions des noms de propositions de deux sous-formules, vous pouvez utiliser la fonction `List.sort_uniq : ('a -> 'a -> int) -> 'a list -> 'a list`. Par exemple `List.sort_uniq compare ["b"; "a"; "b"; "d"]` donne `["a"; "b"; "d"]`.

Exercice 3 : Appelons *interprétation* toute liste d'association de type `(string * bool) list`. Une interprétation permet de spécifier les valeurs de vérité des propositions d'une formule. Chaque proposition apparaît au plus une fois dans la liste.

Pour la formule `And (Prop "p", Prop "q")`, par exemple, une interprétation possible est `[("p", true); ("q", false)]`.

Écrire une fonction `eval_formula : formula -> (string * bool) list -> bool` calculant la valeur de vérité d'une formule relativement à une interprétation donnée.

Par exemple `eval_formula (And (Prop "p", Prop "q")) [("p", true); ("q", false)]` donnera `false`.

Vous pouvez utiliser `List.assoc v l` qui renvoie la partie droite `b` du premier couple `(v, b)` dans la liste `l`, et déclenche l'exception `Not_found` si un tel couple n'existe pas.

Une formule de la logique propositionnelle est appelée une *tautologie* si elle s'évalue à `true` pour toute interprétation spécifiant les valeurs de vérité de ses propositions. Une formule qui s'évalue à `true` pour au moins une interprétation est dite *satisfaisable*.

Exercice 4 : Écrire une fonction `add_to_all : 'a -> ('a list) list -> ('a list) list` telle que `add_to_all x [l1; ...; ln]` renvoie la liste de listes `[x::l1; ...; x::ln]`. Noter que cette fonction est très simple à écrire avec `List.map`.

Exercice 5 : En vous servant de la fonction `add_to_all`, écrire une fonction `interpretations_props : string list -> ((string * bool) list) list` qui, étant donnée une liste de noms de propositions, construit la liste de toutes les interprétations possibles associées.

Par exemple, `interpretations_props ["q"]` donnera `[["q", false]; ["q", true]]` et `interpretations_props ["p"; "q"]` donnera

```
[[("p", false); ("q", false)]; [("p", false); ("q", true)];
[("p", true); ("q", false)]; [("p", true); ("q", true)]].
```

En déduire une fonction `interpretations : formula -> (string * bool) list` qui étant donnée une formule donne une liste de toutes les interprétations associées à ses propositions.

Exercice 6 : À l'aide de la fonction `interpretations`, de `List.exists` et de `List.for_all`, écrire `satisfiable : formula -> bool` qui, étant donnée une formule, détermine si elle est satisfaisable, et `tautology : formula -> bool` qui, étant donnée une formule, détermine si elle est une tautologie. Par exemple,

`satisfiable (And (Prop "p", Neg (Prop "p")))` donnera `false`, et
`tautology (Or (Prop "p", Neg (Prop "p")))` donnera `true`.

Exercice 7 : Si F et G sont deux formules de la logique propositionnelle, on dit que G est *conséquence logique* de F si, pour toute interprétation pour laquelle F s'évalue à *true*, G s'évalue aussi à *true*. Les interprétations considérées doivent spécifier à la fois les valeurs des propositions de F et de celles de G . Les formules F et G sont *équivalentes* si G est conséquence logique de F et F est conséquence logique de G . Écrire une fonction `is_consequence : formula -> formula -> bool` qui vérifie si son deuxième argument est conséquence logique du premier. En déduire une fonction `are_equivalent : formula -> formula -> bool` qui détermine si ses deux arguments sont des formules équivalentes.

Exercice 8 : Dans la logique propositionnelle, les propriétés suivantes sont vérifiées :

1. *Descente des négations.* Les formules suivantes sont équivalentes :
 - $\neg\neg F$ et F ;
 - $\neg(F \vee G)$ et $\neg F \wedge \neg G$;
 - $\neg(F \wedge G)$ et $\neg F \vee \neg G$.
2. *Descente des disjonctions.* Les formules suivantes sont équivalentes :
 - $F \vee (G \wedge H)$ et $(F \vee G) \wedge (F \vee H)$;
 - $(F \wedge G) \vee H$ et $(F \vee H) \wedge (G \vee H)$.

Ces équivalences peuvent se lire comme des règles qui permettent de simplifier une formule en remplaçant des occurrences des formules de gauche par les formules de droite correspondantes. Par exemple, à partir de $\neg(\neg F \vee G)$, on obtient $F \wedge \neg G$ en appliquant d'abord la deuxième règle, puis la première.

Un *littéral* est soit une proposition, soit la négation d'une proposition. Une formule est dite *sous forme normale conjonctive* si elle est de la forme $F_1 \wedge \dots \wedge F_n$ et, pour tout $1 \leq i \leq n$, F_i est de la forme $l_1 \vee \dots \vee l_{n_i}$, où, pour tout $1 \leq j \leq n_i$, l_j est un littéral. Un exemple de formule sous forme normale conjonctive est $(p \vee q \vee \neg r) \wedge (\neg p \vee r) \wedge (\neg q \vee \neg r)$, où p , q et r sont des propositions.

À l'aide des propriétés ci-dessus, écrire les fonctions suivantes :

- `desc_neg : formula -> formula`, qui applique les règles de descente des négations à son argument, jusqu'à ce qu'aucune des trois règles ne s'applique. La formule résultante est dite *sous forme normale de négation*.
- `desc_or : formula -> formula`, qui applique les règles de descente des disjonctions à son argument, supposé en forme normale de négation, jusqu'à ce qu'aucune des deux règles ne s'applique.
- `cnf : formula -> formula`, qui transforme toute formule en une formule équivalente sous forme normale conjonctive.